Contrôle décentralisé d'un essaim de robots par réseaux de neurones cellulaires

CeNN-Inspired Swarm Robotics

12 octobre 2026

Résumé

Ce mémoire étudie le contrôle décentralisé d'un essaim de robots simulés par un réseau de neurones sur graphes partagé. Chaque robot ne traite que son état local, les messages de ses voisins proches et une consigne commune de forme. L'objectif est de former des nuages de points représentant des formes géométriques et symboliques, tout en examinant le comportement face aux pannes et aux changements de taille de l'essaim. La contribution principale est une chaîne expérimentale reproductible : simulateur 2D, jeu de formes, contrôleur GNN, entraînement différentiable et protocole d'évaluation. Un premier protocole pilote montre que le contrôleur appris obtient une meilleure adéquation à une cible circulaire qu'une règle locale de regroupement. Cette observation reste exploratoire : la baseline ne reçoit pas la même information de consigne et l'étude finale devra augmenter le nombre de répétitions et inclure des comparaisons plus équitables.
Mots-clés : robotique en essaim, contrôle décentralisé, réseaux de neurones sur graphes, formation de formes, apprentissage différentiable.

Abstract

This thesis studies decentralized control of a simulated robot swarm with a shared graph neural network. Each robot only processes its local state, messages from nearby neighbours, and a common shape instruction. The main contribution is a reproducible experimental pipeline combining a two-dimensional simulator, a shape dataset, a local GNN controller, differentiable training, and an evaluation protocol. Preliminary experiments show that the learned controller is closer to a circular target than a local gathering rule. This observation remains exploratory because the baseline does not receive the same target instruction and the final study must use more repetitions and information-equivalent comparisons.

Keywords : swarm robotics, decentralized control, graph neural networks, shape formation, differentiable learning.

Table des matières

Résumé **1**

Abstract **2**

1 Introduction **8**
1.1 Contexte 8
1.2 Problématique et question de recherche 9
1.3 Objectifs et contributions 10
1.4 Hypothèses et périmètre 10
1.5 Choix du problème étudié 11
1.6 Critères de réussite 11
1.7 Démarche générale du mémoire 12
1.8 Organisation du mémoire 13

2 État de l'art **14**
2.1 Robotique en essaim et contrôle décentralisé 14
2.2 Flocking et comportements locaux 15
2.3 Formation de formes : de la géométrie imposée à la consigne partagée . . . 15
2.4 Réseaux de neurones sur graphes et passage de messages 16
2.5 Induction, partage des paramètres et changement d'échelle 17
2.6 Apprentissage de contrôleurs décentralisés par GNN 18
2.7 Lecture critique des comparaisons 19
2.8 Réseaux cellulaires et positionnement du mémoire 19

3 Méthode **21**
3.1 Simulateur et hypothèses 21
3.2 Formes cibles 21
3.3 Contrôleur GNN 22
3.4 Apprentissage 23
3.5 Reproductibilité et baselines 24
3.6 Organisation de l'implémentation 24
3.7 Restitution des trajectoires 24

3.8 Informations locales et information de mission 25
3.9 Configurations d'entraînement 25
3.10 Mesures utilisées pendant et après l'apprentissage 26
3.11 Déroulement détaillé d'un épisode 26

4 Expériences et résultats **28**
4.1 Protocole pilote 28
4.2 Résultats préliminaires 29
4.3 Effet du nombre de robots 31
4.4 Lecture des scénarios de panne 32
4.5 Curriculum de formes 32
4.6 Exemples, résultats négatifs et limites de lecture 33
4.7 Discussion des résultats 35
4.8 Lien entre agrégation et résultats observés 36
4.9 Analyse des métriques 36
4.10 Menaces à la validité 37

5 Conclusion et perspectives **38**

Table des figures

1.1 Vue d'ensemble de la chaîne expérimentale. Une perte globale est utilisée pendant l'apprentissage ; lors de l'exécution, chaque robot applique une politique partagée à partir de son observation locale, de ses voisins et de la consigne de mission. 13

2.1 Principe d'agrégation employé dans le contrôleur. Les messages individuels sont calculés pour chaque voisin, puis leur moyenne est utilisée pour mettre à jour l'état local du robot. 17

3.1 Schéma d'un essaim. Le robot orange ne peut échanger qu'avec les robots situés dans sa portée de communication ; les robots distants ne sont pas accessibles directement. 22
3.2 Calcul local réalisé pour un robot lors d'un tour de passage de messages. Les mêmes blocs de calcul et les mêmes paramètres sont partagés dans l'essaim. 23

4.1 Évolution de la perte pendant l'entraînement initial du cercle, pour la graine 41. Cette courbe sert de diagnostic d'optimisation et non d'intervalle de confiance. 29
4.2 Synthèse du protocole pilote calculée à partir de 240 épisodes. À gauche, l'effet de la taille de l'essaim sans panne ; à droite, l'effet des pannes avec 20 robots. Une valeur plus faible de distance de Chamfer indique une meilleure adéquation à la cible circulaire. 30
4.3 Configuration finale obtenue pour l'entraînement du cercle. Les robots représentent l'approximation discrète de la cible par l'essaim. 30
4.4 Distance moyenne de Chamfer en fonction du taux de pannes pour 20 robots dans le protocole pilote. 32
4.5 Distance moyenne de Chamfer lors du passage à l'échelle dans le protocole pilote. Cette figure doit être complétée par les valeurs numériques et leur dispersion dans la campagne finale. 33

4.6 Planche de visualisation issue de la dernière étape du curriculum réduit. Elle illustre des formations approximatives et ne constitue pas encore une validation de lisibilité des symboles. 34

Liste des tableaux

4.1 Résultats du protocole pilote à 20 robots (10 graines par condition). 29
4.2 Passage à l'échelle sans panne dans le protocole pilote, avec dix épisodes par condition. 31

Chapitre 1

Introduction

1.1 Contexte

La robotique en essaim étudie la coopération d'un grand nombre d'agents relativement simples. Plutôt que de confier la mission à un robot chef, elle cherche à faire émerger un comportement collectif à partir de décisions locales, de communications limitées et de règles partagées [2]. Cette organisation présente plusieurs intérêts : elle répartit la charge de calcul, évite un point unique de défaillance et peut, en principe, tolérer l'ajout ou la perte de quelques unités. Elle est pertinente pour des tâches telles que l'exploration, la surveillance distribuée, le transport coopératif ou l'occupation d'une zone.

Ce potentiel s'accompagne toutefois d'une difficulté fondamentale. Le résultat recherché est global — par exemple couvrir une région, maintenir une formation ou dessiner une figure — alors que chaque robot ne dispose que d'une perception partielle de la situation. Les règles classiques de comportement collectif, inspirées notamment du flocking, combinent cohésion, séparation et alignement pour produire des mouvements de groupe plausibles [11, 10]. Elles restent précieuses par leur simplicité et leur interprétabilité, mais leur conception devient délicate lorsque la tâche exige une géométrie précise ou un ensemble varié de cibles.

La formation de formes constitue un cas d'étude particulièrement révélateur. Rassembler des robots vers une même zone ne garantit ni une couverture homogène, ni l'apparition d'un cercle, d'une lettre ou d'un symbole. Pour atteindre une forme, le contrôleur doit simultanément guider les robots vers le contour demandé, éviter les rapprochements excessifs et conserver une distribution suffisante sur l'ensemble de la cible. Des contrôleurs décentralisés spécialisés ont été proposés pour la génération de courbes ou de formations progressives [7, 9]. Leur existence montre la faisabilité du contrôle local, mais aussi la nécessité de définir soigneusement les informations disponibles et les mécanismes de décision.

Dans ce mémoire, la règle locale n'est pas entièrement écrite à la main. Elle est repré-

sentée par un réseau de neurones sur graphes (*Graph Neural Network*, GNN). Le graphe encode naturellement un essaim : les robots sont les nœuds et les liens de communication entre voisins sont les arêtes. Une même politique paramétrée est partagée par tous les robots ; elle agrège les messages provenant du voisinage puis produit une commande de vitesse. Les GNN constituent ainsi une piste adaptée pour apprendre une loi locale tout en conservant une structure d'exécution distribuée [5, 1, 12].

1.2 Problématique et question de recherche

Le contrôle centralisé fournit une solution conceptuellement simple à la formation de formes : un planificateur connaît les positions de tous les robots, associe éventuellement chaque robot à un point de la cible, puis transmet les commandes. Cette hypothèse est peu compatible avec un essaim dont les échanges sont limités par la portée radio, dont la taille varie et dont certains agents peuvent disparaître. Elle masque également la question principale de la robotique en essaim : comment un résultat collectif peut-il être obtenu lorsqu'aucun agent ne possède directement l'état complet du groupe ?

Le présent travail étudie donc la question suivante :

> *Un réseau de neurones sur graphes, exécuté à partir d'informations locales et d'une consigne commune de forme, peut-il apprendre à former des configurations spatiales et conserver un comportement utile en présence de pannes ou d'un changement de taille de l'essaim ?*

Cette formulation appelle trois précisions. Premièrement, le caractère décentralisé concerne la *décision de mouvement* : chaque robot applique le même contrôleur et n'échange qu'avec ses voisins à portée. Deuxièmement, la forme à atteindre est une consigne de mission partagée avant l'exécution ; elle n'est pas découverte spontanément par l'essaim. Selon la configuration expérimentale, cette consigne prend la forme d'un identifiant de cible ou d'une information géométrique locale relative au contour. Enfin, l'entraînement est centralisé dans un simulateur différentiable : une perte globale peut y être calculée pour ajuster les poids, alors que la politique déployée ne reçoit pas les positions des robots distants.

La question générale est déclinée en quatre sous-questions :

1. Le contrôleur appris réduit-il l'écart entre la configuration finale et une forme cible à partir de positions initiales aléatoires ?
2. L'échange de messages entre voisins apporte-t-il un bénéfice par rapport à des règles locales ou à une politique sans communication, à information de consigne comparable ?
3. La politique à paramètres partagés reste-t-elle applicable lorsque le nombre de robots et le nombre de voisins changent ?

4. Comment la qualité de la formation se dégrade-t-elle lorsque des robots sont retirés pendant un épisode ?

1.3 Objectifs et contributions

L'ambition du mémoire n'est pas de valider immédiatement une flotte de robots physiques. Il s'agit d'abord de construire un banc d'essai cohérent et reproductible permettant d'étudier une hypothèse de contrôle par GNN. Les principales contributions du travail sont les suivantes :

— un simulateur bidimensionnel qui représente la position, la vitesse, le voisinage de communication, les limites de l'arène et la perte de robots ;
— un jeu de formes converties en nuages de points normalisés, comprenant des géométries, des chiffres, des lettres et des nombres ;
— un contrôleur GNN à passage de messages, partagé entre tous les robots et compatible avec un voisinage de taille variable ;
— une procédure d'apprentissage direct à travers le simulateur, complétée par un curriculum allant de formes simples vers des formes symboliques ;
— un protocole d'évaluation qui mesure la proximité à la cible, les collisions, les pannes et le passage à l'échelle, tout en séparant les résultats pilotes des conclusions définitives.

Le titre déposé emploie l'expression « réseaux de neurones cellulaires ». L'analogie est justifiée par le fait que tous les robots appliquent la même règle et n'interagissent qu'avec un voisinage limité, à la manière d'unités cellulaires. L'implémentation réalisée est toutefois un GNN à passage de messages sur un graphe dynamique. Dans la suite du document, le terme GNN est donc utilisé pour décrire précisément l'architecture ; toute modification officielle de l'intitulé reste soumise à l'accord de l'encadrement.

1.4 Hypothèses et périmètre

Les expériences sont réalisées dans une arène plane sans obstacle fixe. Chaque robot possède une position et une vitesse en deux dimensions ; les déplacements sont représentés par une dynamique simplifiée à vitesse directement commandée. Deux robots peuvent communiquer lorsque leur distance est inférieure à une portée définie. Les mesures et les messages sont supposés exacts, instantanés et sans perte. Un robot déclaré en panne est retiré de la dynamique active et du graphe de communication.

Ces choix permettent d'isoler l'effet de l'architecture de contrôle et de l'apprentissage. Ils excluent volontairement plusieurs problèmes importants : estimation de pose distribuée, incertitude des capteurs, délais de communication, obstacles, dynamique réaliste

des actionneurs, consommation énergétique et validation sur matériel. Le projet ne prétend donc pas démontrer un transfert direct vers des robots réels. Ces limites constituent au contraire des axes de discussion et de prolongement.

1.5 Choix du problème étudié

Le choix de la formation de formes répond à une raison méthodologique. Il est suffisamment simple pour être représenté dans un simulateur 2D et visualisé directement, mais il reste plus exigeant qu'un simple rassemblement. Dans un rassemblement, plusieurs robots peuvent finir dans une même zone tout en donnant l'impression que la tâche est accomplie. Dans une formation, ce comportement est insuffisant : les robots doivent aussi se répartir sur les différentes parties de la cible. Cette exigence rend visibles plusieurs difficultés de contrôle distribué, notamment la couverture, les collisions et les ruptures de connectivité.

Le cercle sert de premier cas de validation. Sa géométrie est régulière et son contour peut être échantillonné facilement. Les chiffres, les lettres et les nombres permettent ensuite de vérifier que le pipeline ne dépend pas exclusivement d'une forme convexe et symétrique. Leur intérêt n'est pas de faire de la reconnaissance de caractères. Ils fournissent plutôt des contours présentant des segments, des angles, des régions moins régulières et parfois plusieurs parties visuellement distinctes. Une lettre formée de manière approximative reste donc un résultat de contrôle spatial, non la preuve qu'un système comprend un symbole.

Le projet adopte par ailleurs une vision pragmatique de la décentralisation. Il serait trompeur d'affirmer qu'un robot ne reçoit aucune information commune : toute mission suppose au minimum un objectif partagé. Ici, la forme à réaliser est connue avant le début de l'épisode. En revanche, la décision instantanée de mouvement ne dépend pas d'un tableau central contenant toutes les positions de l'essaim. Cette distinction entre une consigne commune et une observation globale est importante pour interpréter les résultats.

1.6 Critères de réussite

Une expérience ne sera pas considérée comme réussie sur la seule base d'une animation convaincante. La première condition est que les robots se rapprochent du nuage de points cible tout en couvrant ses différentes zones. La seconde est que les commandes restent bornées et que les rapprochements dangereux soient peu fréquents. La troisième concerne la reproductibilité : une même configuration et une même graine doivent permettre de retrouver le même scénario, tandis que plusieurs graines doivent être utilisées pour éviter de conclure à partir d'un cas favorable.

Les critères liés à la robustesse doivent être évalués séparément. Tester plus de robots que lors de l'entraînement ne prouve pas à lui seul que le modèle est généralisable ; cela montre seulement qu'il peut être exécuté avec une taille différente. De même, retirer des robots teste une panne simplifiée, mais ne représente pas encore des capteurs erronés ou un robot immobilisé qui deviendrait un obstacle. Cette manière de préciser les critères de réussite permet de limiter les conclusions aux expériences effectivement réalisées.

1.7 Démarche générale du mémoire

La démarche suivie part d'un problème simple et augmente progressivement la difficulté. Il faut d'abord disposer d'un simulateur qui produit un voisinage cohérent et une dynamique stable. Sans cette étape, une mauvaise formation pourrait provenir du calcul des distances, de la mise à jour des positions ou de la visualisation, et non du modèle appris. Une règle locale de regroupement est donc utilisée comme contrôle initial. Elle n'est pas présentée comme la solution au problème de formation de formes ; elle sert à vérifier le socle de simulation.

La deuxième étape consiste à représenter la forme cible. Le choix d'un nuage de points permet de traiter une forme comme un ensemble de positions à couvrir plutôt que comme une image dont le réseau devrait prédire chaque pixel. Cette représentation est compatible avec un nombre limité de robots et permet d'utiliser une distance entre ensembles de points. Elle rend aussi explicite une contrainte pratique : si une forme comporte plus de détails que le nombre de robots disponibles, certains détails ne pourront pas être rendus correctement.

La troisième étape est l'apprentissage de la politique GNN. L'objectif n'est pas de produire une trajectoire prédéfinie pour un robot particulier. Les robots commencent dans des positions aléatoires et appliquent tous les mêmes paramètres. L'apprentissage doit donc favoriser une loi qui réagit aux configurations locales rencontrées pendant les épisodes. Ce choix explique l'importance des graines, des scénarios variés et des évaluations séparées de l'entraînement.

Enfin, les résultats sont analysés à plusieurs niveaux. Une courbe de perte renseigne sur l'optimisation. Une image ou une animation renseigne sur la qualité géométrique observable. Des agrégats sur plusieurs épisodes renseignent sur la variabilité. Aucun de ces niveaux ne suffit isolément. Le mémoire adopte cette séparation pour éviter qu'une animation spectaculaire soit interprétée comme une validation statistique, ou qu'une moyenne favorable masque des échecs importants.

1.8 Organisation du mémoire

Le chapitre 2 situe la formation décentralisée dans la robotique en essaim et présente le rôle des GNN dans le contrôle distribué. Le chapitre 3 décrit le simulateur, les formes cibles, l'architecture du contrôleur, l'apprentissage et les choix de reproductibilité. Le chapitre 4 présente les expériences, les résultats disponibles et leur discussion. Enfin, le chapitre 5 synthétise les apports et les perspectives.

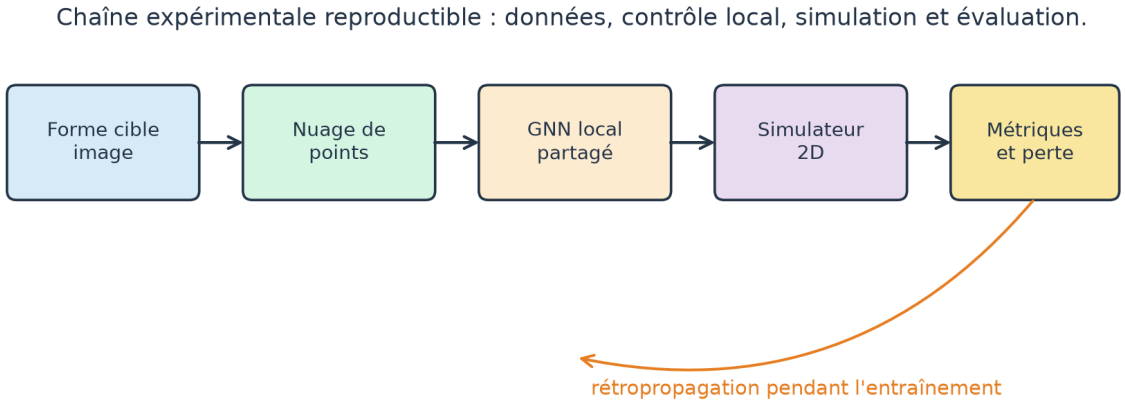


FIGURE 1.1 – Vue d'ensemble de la chaîne expérimentale. Une perte globale est utilisée pendant l'apprentissage ; lors de l'exécution, chaque robot applique une politique partagée à partir de son observation locale, de ses voisins et de la consigne de mission.

Chapitre 2

État de l'art

2.1 Robotique en essaim et contrôle décentralisé

La robotique en essaim s'intéresse à la coordination d'agents nombreux, homogènes ou faiblement hétérogènes, qui accomplissent collectivement une tâche sans dépendre d'un superviseur unique. La revue de Brambilla et al. souligne que cette approche associe généralement simplicité individuelle, redondance et interactions locales [2]. L'objectif n'est pas seulement de répartir une tâche entre plusieurs robots : il est de concevoir des règles locales dont l'effet collectif reste utile lorsque la taille du groupe, la topologie de communication ou la disponibilité des agents évoluent.

Les modèles comportementaux ont fourni des exemples fondateurs de cette idée. Le modèle de Reynolds repose sur trois mécanismes simples — séparation, alignement et cohésion — qui génèrent des déplacements collectifs crédibles sans plan global explicite [11]. Dans un cadre de contrôle multi-agent, Olfati-Saber formalise des lois de flocking qui combinent cohésion, évitement de collision et maintien de la connectivité à travers un graphe d'interactions [10]. Ces approches ont l'avantage de rendre les mécanismes de décision lisibles : un terme de répulsion produit un effet identifiable, de même qu'un terme d'attraction vers les voisins.

Leur limite est que le concepteur doit choisir la structure de la loi, les termes à combiner et les paramètres numériques. Une règle de regroupement peut amener les robots vers une zone commune, sans pour autant les répartir sur un contour précis. De la même manière, une règle bien réglée pour un cercle ne se généralise pas automatiquement à une lettre ou à un nombre. La formation de formes déplace donc le problème de la simple cohésion vers la construction distribuée d'une géométrie donnée.

2.2 Flocking et comportements locaux

Le flocking désigne un déplacement collectif dans lequel les agents restent groupés tout en conservant une direction de mouvement cohérente. Dans sa forme classique, il combine trois idées. La cohésion rapproche un agent du centre de ses voisins. La séparation produit une force opposée lorsque deux agents deviennent trop proches. L'alignement rapproche la vitesse d'un agent de la vitesse moyenne observée autour de lui. Ces mécanismes ne nécessitent pas de chef, car chaque robot applique la même règle à partir de son voisinage [11].

Cette famille de règles est une référence utile pour deux raisons. D'abord, elle donne un comportement de base facilement interprétable : si les robots s'accumulent, on peut renforcer la séparation ; s'ils se dispersent, on peut renforcer la cohésion. Ensuite, elle montre que des interactions très simples suffisent à créer un comportement collectif. Dans ce projet, la règle de cohésion-répulsion joue ce rôle de contrôle : elle permet de vérifier que le simulateur et le calcul de voisinage produisent bien un regroupement avant d'introduire le GNN.

Le flocking ne résout toutefois pas directement la formation de forme. Un essaim peut rester cohérent et se déplacer de manière alignée tout en restant loin du contour demandé. Ajouter une attraction vers une cible est possible, mais il faut alors définir comment chaque robot choisit une zone du contour et comment éviter que tous convergent vers le même endroit. C'est précisément l'espace de décision que l'apprentissage cherche à explorer dans ce mémoire. Le GNN ne remplace pas les idées de cohésion ou de séparation par magie ; il apprend une fonction de décision qui peut utiliser les vitesses, les distances et les messages du voisinage pour réduire la perte définie pendant l'entraînement.

2.3 Formation de formes : de la géométrie imposée à la consigne partagée

La formation de formes vise à faire converger les positions des robots vers une structure spatiale : une courbe, une formation rigide, une surface ou un nuage de points. Hsieh, Kumar et Chaimowicz étudient des contrôleurs décentralisés pour la génération de formes par essaims et mettent en évidence le rôle de l'information géométrique disponible localement [7]. Li et al. proposent une approche progressive de formation de formes, qui cherche à construire la cible à partir de décisions distribuées [9]. Ces travaux montrent que la forme elle-même doit être représentée d'une manière compatible avec les observations des agents.

Plusieurs représentations sont envisageables. Une cible peut être décrite par une équation analytique, par des positions de référence attribuées aux robots, par une carte, par des points de contour ou par des informations perceptives. Chaque choix modifie la difficulté du problème. Une affectation fixe robot–point fournit une information très forte,

mais impose de résoudre l'association et peut limiter l'adaptation aux pannes. À l'inverse, une simple étiquette de forme contient peu de géométrie : elle est facile à partager, mais elle demande au contrôleur d'apprendre comment cette étiquette doit modifier le comportement.

Le présent projet utilise des nuages de points comme représentation de la cible. Cette représentation évite de lier durablement un robot particulier à un point particulier ; la qualité de formation est mesurée par la proximité entre les deux ensembles de points. Elle doit cependant être interprétée correctement. Le nuage cible est une consigne de mission définie avant le déploiement ; son accès n'est pas une communication entre robots. Lorsque la configuration utilise le vecteur vers le point de contour le plus proche, elle suppose une forme de carte ou de requête géométrique locale. Cette hypothèse facilite la formation de contours complexes, mais elle ne doit pas être confondue avec une perception autonome de la forme dans un environnement réel.

2.4 Réseaux de neurones sur graphes et passage de messages

Un essaim à voisinage local se modélise naturellement par un graphe dynamique $\mathcal{G}^t = (\mathcal{V}, \mathcal{E}^t)$. Chaque robot est un nœud ; une arête relie deux robots lorsqu'ils peuvent échanger des informations au pas de temps t. Les réseaux de neurones sur graphes étendent les réseaux neuronaux à ce type de données relationnelles. Dans le cadre du passage de messages, un nœud produit ou reçoit des messages calculés à partir des états des nœuds reliés et des attributs d'arête, puis agrège ces messages avant de mettre à jour sa représentation [5]. Le cadre plus général des *graph networks* formalise de la même façon les mises à jour de nœuds, d'arêtes et, éventuellement, d'attributs globaux [1].

Deux propriétés rendent ce formalisme particulièrement adapté au contrôle d'essaim. D'abord, les paramètres sont partagés : la même fonction de message et la même fonction de décision peuvent être utilisées pour chaque robot. Ensuite, une agrégation par somme, moyenne ou maximum est invariante à l'ordre des voisins. Un robot peut donc recevoir un nombre différent de messages au cours du temps sans qu'il soit nécessaire de modifier la taille de l'entrée du réseau. Ces propriétés ne garantissent pas à elles seules une bonne généralisation, mais elles introduisent un biais inductif cohérent avec le caractère relationnel et variable d'un essaim.

Dans le contrôleur implémenté, l'agrégation est une moyenne. Chaque voisin j envoie d'abord un message m_{ij} calculé à partir de son état, de l'état du robot i et de leur géométrie relative. Le robot i calcule ensuite :

$$\bar{m}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} m_{ij}. \tag{2.1}$$

La moyenne a une conséquence pratique : la magnitude de l'information agrégée ne croît pas mécaniquement avec le nombre de voisins, contrairement à une somme non normalisée. Un robot au centre d'un groupe dense et un robot proche du bord peuvent ainsi être traités avec des échelles numériques plus comparables. Lorsqu'un robot n'a aucun voisin, l'agrégat est remplacé par un vecteur nul, puis le réseau produit tout de même une commande valide à partir de son état propre et de la consigne.

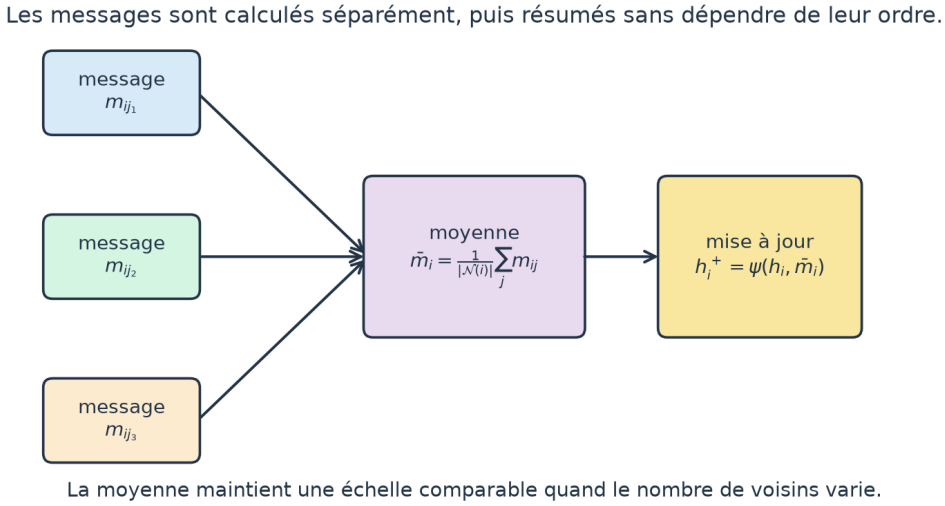


FIGURE 2.1 – Principe d'agrégation employé dans le contrôleur. Les messages individuels sont calculés pour chaque voisin, puis leur moyenne est utilisée pour mettre à jour l'état local du robot.

Le passage de messages possède également une limite structurelle. Avec L tours d'échange, l'état d'un robot ne peut intégrer directement que les informations disponibles dans son voisinage à au plus L sauts, pour la topologie considérée. Augmenter L peut étendre la portée informationnelle, mais augmente aussi le coût de calcul et peut rendre l'apprentissage plus difficile. Un GNN ne transforme donc pas un système local en contrôleur centralisé : il fournit une manière paramétrable d'exploiter des communications restreintes.

2.5 Induction, partage des paramètres et changement d'échelle

La possibilité d'appliquer un même modèle à une taille d'essaim différente est souvent associée au caractère inductif des architectures de graphe. Dans GraphSAGE, Hamilton, Ying et Leskovec apprennent une fonction qui agrège les attributs d'un voisinage local au lieu d'apprendre un vecteur propre à chaque nœud [6]. Le problème traité dans cet article est différent de la robotique, mais le principe est utile ici : une politique ne doit pas dépendre de l'identifiant d'un robot ni d'une liste d'entrée de longueur fixe.

Cette propriété ne doit pas être présentée comme une garantie de performance. Lorsque le nombre de robots augmente, la densité locale, la connectivité et le nombre moyen de voisins peuvent aussi changer. Une politique peut donc être techniquement exécutable avec cent robots tout en produisant une formation moins régulière que pour vingt robots. Le passage à l'échelle doit être mesuré, avec une cible et des conditions initiales adaptées, plutôt que supposé à partir de l'architecture.

Les travaux sur les contrôleurs GNN décentralisés insistent également sur cet écart entre propriété architecturale et validation expérimentale. Gama et al. proposent d'apprendre des contrôleurs sur des graphes en exploitant le partage des paramètres et l'invariance aux permutations, puis évaluent le transfert vers des équipes plus grandes et des scénarios nouveaux [4]. Dans le cas de la formation triangulaire étudiée par Jiang et al., une politique entraînée avec cinq robots est testée avec des équipes de quatre à neuf robots [8]. Ces résultats motivent les essais de passage à l'échelle du présent projet, mais ne dispensent pas d'un protocole spécifique aux formes et aux hypothèses retenues ici.

2.6 Apprentissage de contrôleurs décentralisés par GNN

Les GNN ont été étudiés comme politiques de contrôle lorsque les agents communiquent sur un graphe. Tolstaya et al. montrent qu'un contrôleur GNN peut être entraîné pour des tâches de robotique en essaim tout en conservant une exécution locale sur un graphe de communication mobile [12]. Gama, Tolstaya et Ribeiro analysent plus largement les GNN comme contrôleurs décentralisés et mettent l'accent sur les propriétés de partage des paramètres et de transfert entre topologies [3]. Ces travaux motivent l'usage d'une architecture relationnelle plutôt qu'un réseau dense dont l'entrée supposerait une liste fixe de robots.

Pour la formation de groupe, Jiang, Huang et Guo proposent une méthode de formation décentralisée de bout en bout fondée sur un GNN [8]. Ce type de travail rapproche directement l'apprentissage du problème étudié ici : un contrôleur doit convertir des observations locales et des informations de voisinage en actions coordonnées. Les approches diffèrent toutefois selon les capteurs disponibles, la forme à produire, le mode d'apprentissage et la manière dont la cible est communiquée. Une comparaison rigoureuse ne peut donc pas se limiter à constater que plusieurs méthodes utilisent un GNN ; elle doit examiner quelles informations sont réellement accessibles à chaque robot au moment de l'exécution.

Deux stratégies d'apprentissage sont courantes. La première consiste à apprendre à imiter un contrôleur expert ou centralisé ; elle peut produire un signal d'entraînement informatif, mais la politique finale doit être évaluée sans accéder à l'information centrale

de l'expert. La seconde consiste à optimiser directement le comportement dans un simulateur différentiable ou par renforcement. Le projet retient ici l'apprentissage direct à travers un simulateur différentiable, afin d'optimiser une mesure de proximité à la forme et des contraintes de sécurité. Cette décision favorise la reproductibilité et la simplicité du prototype ; elle ne constitue pas une comparaison expérimentale entre les paradigmes d'apprentissage.

2.7 Lecture critique des comparaisons

La comparaison avec une méthode classique exige que les méthodes reçoivent des informations comparables. Une règle de cohésion et de répulsion qui ne connaît que les voisins peut être une bonne baseline pour le regroupement, mais elle ne dispose pas de la même information qu'un GNN recevant une consigne de forme. Si le GNN obtient une meilleure distance à une cible circulaire dans cette situation, le résultat établit surtout que la consigne et la politique entraînée sont adaptées à la tâche. Il ne permet pas encore d'isoler l'effet du GNN lui-même.

Une étude plus complète devrait comporter au moins trois contrôles. Le premier est une règle manuelle qui reçoit une description de cible comparable. Le deuxième est un réseau sans communication entre voisins, mais avec la même consigne de forme et une taille de modèle similaire. Le troisième est une variante de profondeur différente, par exemple un ou trois tours de messages. Ces ablations ne sont pas accessoires : elles permettent de répondre à la question précise de l'apport de la communication et de l'architecture relationnelle.

2.8 De la règle écrite à la règle apprise

Une différence importante entre les méthodes classiques et le GNN concerne la place du concepteur. Dans une loi de potentiel, le concepteur écrit explicitement les forces d'attraction, de répulsion et éventuellement d'alignement. Il choisit aussi leurs gains et les distances à partir desquelles elles agissent. Cette démarche apporte de la lisibilité : lorsque le système se comporte mal, il est possible d'identifier un terme et de le modifier. En contrepartie, l'ajout d'une forme complexe ou d'une nouvelle contrainte peut demander une nouvelle dérivation de la loi et plusieurs réglages manuels.

Dans une approche apprise, le concepteur ne choisit pas directement la réaction exacte à chaque configuration. Il choisit les informations accessibles, la structure du réseau, les objectifs à optimiser et les scénarios d'entraînement. Les paramètres de la politique sont ensuite ajustés à partir des épisodes simulés. Le déplacement de responsabilité est réel : une mauvaise observation locale, une perte mal définie ou un entraînement trop court peuvent empêcher le modèle d'apprendre, même si le GNN est bien implémenté.

Il serait toutefois simpliste d'opposer une méthode explicable à une méthode opaque. Les deux familles dépendent d'hypothèses de modélisation. Une règle de flocking suppose qu'attraction, répulsion et alignement sont suffisants pour la mission choisie. Le GNN suppose que les exemples rencontrés pendant l'entraînement couvrent les situations utiles et que la perte favorise réellement la formation souhaitée. Le mémoire ne cherche donc pas à déclarer une famille universellement meilleure. Il étudie dans quelles conditions une règle apprise, exécutée localement, peut être une alternative raisonnable à une règle manuelle.

2.9 Consigne, perception et déploiement

Les publications sur le contrôle décentralisé ne font pas toutes les mêmes hypothèses sur les capteurs. Certaines utilisent les positions absolues des robots, d'autres des mesures de distance et d'angle, d'autres encore des cartes d'occupation issues de capteurs embarqués. Jiang et al. utilisent par exemple des observations LiDAR transformées en cartes locales, alors que le prototype de ce mémoire travaille avec des états simulés plus simples [8]. Cette différence doit être prise au sérieux : une architecture qui fonctionne avec une position exacte ne garantit pas qu'elle fonctionnera après remplacement par un capteur bruité.

Le choix du présent projet est intermédiaire. La communication entre robots est locale, mais la forme cible est fournie comme une consigne partagée. Cette hypothèse est justifiée pour une mission préparée à l'avance, par exemple une formation lumineuse ou une distribution planifiée. Elle serait moins adaptée à une tâche où la forme doit être découverte dans l'environnement. Dans ce second cas, il faudrait intégrer une perception locale, une carte distribuée ou un mécanisme de consensus sur la cible.

Cette distinction permet aussi de préciser le sens du mot décentralisé. Un contrôleur peut être décentralisé au moment de l'exécution tout en ayant été entraîné de manière centralisée. Il peut aussi utiliser une consigne commune sans recevoir les états distants de tous les robots. Ce qui serait contraire à l'hypothèse de travail serait de calculer, à chaque pas, une affectation globale des robots à la cible et de la transmettre à chacun. Le prototype évite cette opération pendant l'exécution, mais la consigne géométrique reste une hypothèse forte qui devra être discutée dans toute extension vers le réel.

2.10 Réseaux cellulaires et positionnement du mémoire

L'intitulé initial du mémoire fait référence aux réseaux de neurones cellulaires. L'idée commune est locale : des unités identiques évoluent selon leurs interactions avec un voisinage limité. Dans l'essaim étudié, le voisinage n'est pas une grille fixe ; il dépend des

positions et de la portée de communication. De plus, les messages sont produits par un réseau appris. L'implémentation est donc plus précisément décrite comme un GNN à passage de messages. La référence aux réseaux cellulaires doit être comprise comme une inspiration de contrôle local homogène, non comme l'implémentation littérale d'un réseau cellulaire classique.

Le positionnement de ce mémoire est expérimental. Il ne revendique pas une nouvelle théorie de la formation distribuée ni une supériorité générale du GNN. Il met en place une chaîne reproductible qui relie un simulateur 2D, des formes cibles, une politique partagée, un apprentissage différentiable et des mesures de robustesse. La contribution recherchée est double : vérifier qu'une telle politique peut former des cibles dans un cadre contrôlé, puis identifier les conditions qui limitent cette capacité.

Cette prudence implique deux exigences pour les expériences. Premièrement, les baselines doivent recevoir une information de consigne comparable à celle du GNN. Une règle de simple regroupement constitue un contrôle utile du simulateur, mais ne suffit pas à démontrer un gain dû à l'apprentissage de la forme. Deuxièmement, les résultats visuels doivent être complétés par des métriques, plusieurs graines et une analyse des échecs. Les chapitres suivants décrivent le protocole conçu dans cette perspective.

Chapitre 3

Méthode

3.1 Simulateur et hypothèses

On considère N robots évoluant dans une arène carrée bidimensionnelle. L'état du robot i au pas t comprend sa position $p_i^t \in R^2$ et sa vitesse $v_i^t \in R^2$. Le contrôleur produit une commande bornée u_i^t, puis le simulateur applique une dynamique à vitesse directe :

$$p_i^{t+1} = p_i^t + \Delta t\, u_i^t, \qquad \|u_i^t\| \leq v_{\max}. \tag{3.1}$$

La configuration utilisée pour les premiers entraînements comporte 20 robots dans une arène de 10×10, un pas de temps de 0,3 et une vitesse maximale de 1. Ces valeurs servent à stabiliser l'expérimentation ; elles ne sont pas interprétées comme les paramètres d'une plateforme physique particulière.

Le graphe de communication $\mathcal{G}^t = (\mathcal{V}, \mathcal{E}^t)$ est recalculé à chaque pas. Une arête est présente entre i et j lorsque $\|p_i^t - p_j^t\| \leq r_c$, avec $r_c = 2{,}5$ dans la configuration de référence. Ainsi, la topologie évolue avec les déplacements. Un robot en panne cesse de se déplacer et n'émet plus de messages ; il est retiré des calculs de la politique des robots actifs. Cette première version adopte un repère commun et des mesures parfaites. L'absence de repère propre, de délai ou de bruit constitue une limite discutée au chapitre 4.

3.2 Formes cibles

Le jeu de données contient sept formes géométriques, les 26 lettres de l'alphabet, les dix chiffres et un ensemble de nombres à trois caractères. Chaque image est convertie en un nuage de points puis normalisée dans un cadre commun. Pour une exécution donnée, un sous-ensemble de points de la cible est sélectionné afin de constituer la référence de formation. Cette représentation évite d'imposer une correspondance fixe entre l'indice d'un robot et l'indice d'un point cible.

La consigne disponible à tous les robots comporte un identifiant de forme et le nuage

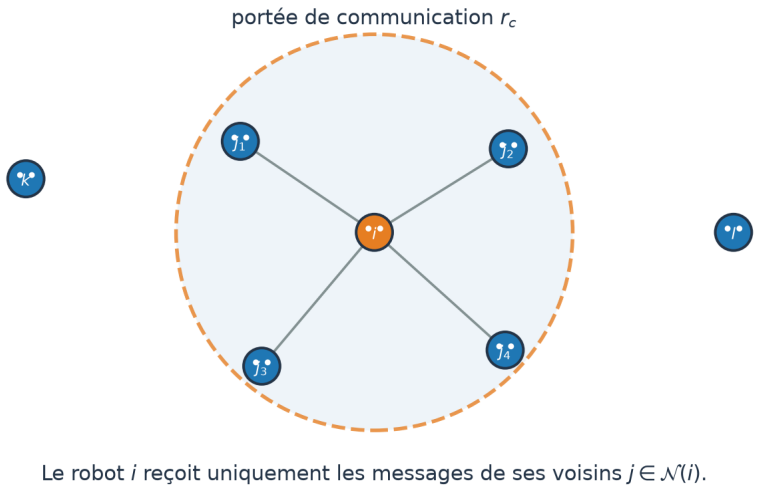


FIGURE 3.1 – Schéma d'un essaim. Le robot orange ne peut échanger qu'avec les robots situés dans sa portée de communication ; les robots distants ne sont pas accessibles directement.

de points associé. À partir de sa propre position, un robot interroge ce nuage et obtient le vecteur vers le point de contour le plus proche. Cette information locale de géométrie est plus riche qu'un identifiant seul : elle indique dans quelle direction se trouve le trait à couvrir, sans fournir la position des autres robots. L'identifiant et le nuage de points ne sont pas des messages provenant d'un superviseur au cours de l'exécution : ils représentent une mission définie avant le déploiement. Le choix d'une consigne géométrique partagée facilite l'étude de la formation, mais il introduit une hypothèse qui devra être remplacée à terme par une perception, une carte ou une balise localement observable.

3.3 Contrôleur GNN

Chaque robot encode sa vitesse, le vecteur g_i vers le contour cible le plus proche et l'identifiant de forme. Son état latent initial est noté h_i^0. À la couche ℓ, le message de j vers i dépend des deux états et de la position relative :

$$m_{ij}^{\ell} = \phi_{\ell}\left(h_i^{\ell}, h_j^{\ell}, p_j - p_i\right), \qquad \bar{m}_i^{\ell} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} m_{ij}^{\ell}. \tag{3.2}$$

Lorsque le voisinage est vide, l'agrégat est remplacé par le vecteur nul. Une fonction de mise à jour ψ_{ℓ} produit ensuite $h_i^{\ell+1} = \psi_{\ell}(h_i^{\ell}, \bar{m}_i^{\ell})$. Deux tours de passage de messages et une dimension cachée de 32 sont employés dans les expériences initiales. Un décodeur partagé transforme l'état final en commande de vitesse, puis une non-linéarité borne cette commande.

Le partage des paramètres garantit que la politique ne dépend pas de l'identité d'un robot. L'agrégation par moyenne la rend également insensible à l'ordre des voisins. En revanche, la portée effective de l'information reste limitée : avec deux tours, un robot ne résume directement que son voisinage à deux sauts pour la topologie considérée à cet instant.

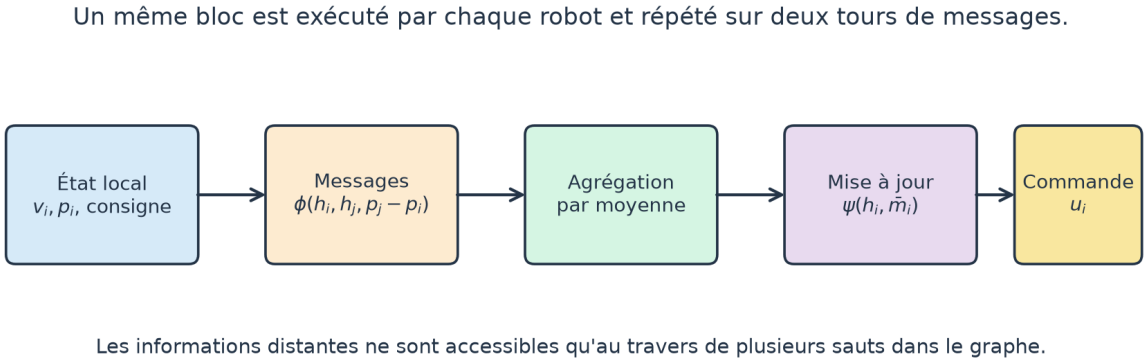


FIGURE 3.2 – Calcul local réalisé pour un robot lors d'un tour de passage de messages. Les mêmes blocs de calcul et les mêmes paramètres sont partagés dans l'essaim.

3.4 Apprentissage

L'entraînement est direct à travers le simulateur différentiable. Si P désigne l'ensemble des positions finales des robots actifs et Q le nuage de points cible, la qualité de forme est mesurée par une distance de Chamfer symétrique :

$$d_C(P, Q) = \frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \|p - q\|^2 + \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|q - p\|^2. \tag{3.3}$$

La fonction optimisée ajoute à ce terme une pénalité lorsque deux robots franchissent une distance de sécurité, une faible pénalité sur l'amplitude des commandes et une régularisation de couverture. Cette dernière compare les statistiques de distances au plus proche voisin de l'essaim à celles de la cible : elle réduit le risque qu'une distance de Chamfer faible soit obtenue par un amas de robots sur une fraction seulement de la forme. Les poids utilisés sont explicitement versionnés dans chaque configuration.

Un curriculum réduit introduit successivement les géométries, les chiffres, quelques lettres, puis deux nombres. Une seconde configuration, dédiée aux formes complexes, porte l'essaim et le nuage cible à 50 points, augmente la durée des épisodes, la profondeur du GNN et le nombre d'itérations. Ce procédé vise à limiter la difficulté initiale de

l'apprentissage, non à établir que le modèle comprend la structure linguistique des symboles. Le meilleur état observé au cours de chaque niveau est sauvegardé pour limiter la dégradation en fin d'entraînement.

3.5 Reproductibilité et baselines

Les paramètres des simulations, de l'entraînement et de l'évaluation sont conservés dans des fichiers de configuration versionnés, avec une graine explicite. Les modèles et tableaux produits sont séparés du code source. La première baseline est une règle locale de cohésion–répulsion : elle fournit un point de comparaison interprétable pour le regroupement, mais ne reçoit pas la consigne circulaire. Elle est donc utile comme contrôle de base, et non comme comparaison informationnellement équivalente avec le GNN.

3.6 Organisation de l'implémentation

La mise en oeuvre sépare volontairement les composants qui relèvent de la simulation de ceux qui relèvent de l'apprentissage. Le module de simulation met à jour les positions, applique les limites de l'arène, construit le voisinage et gère les pannes. Le module de génération de formes produit les images et les nuages de points. Le module de modèle contient le contrôleur GNN, tandis que les modules d'entraînement et d'évaluation exécutent les épisodes, calculent les pertes et enregistrent les résultats.

Cette séparation a un intérêt pratique. Une même condition initiale peut être évaluée avec la règle de regroupement ou avec le GNN sans modifier la dynamique de base. Elle permet également de tester séparément le calcul du voisinage, le respect de la vitesse maximale, le traitement d'un robot isolé et l'invariance au réordonnancement des robots. Les figures ne pilotent pas la simulation : elles sont construites après l'exécution à partir des positions enregistrées.

Les paramètres numériques sont regroupés dans des fichiers de configuration. Ils comprennent notamment la graine aléatoire, le nombre de robots, le rayon de communication, la durée d'un épisode, la vitesse maximale et les poids de chaque terme de la perte. Après un entraînement, le projet sauvegarde le modèle, la configuration utilisée, la liste des pertes et les métriques disponibles. Cette traçabilité ne transforme pas une seule graine en résultat général, mais elle permet de rejouer une expérience, de comparer deux réglages et de repérer l'origine d'une différence.

3.7 Restitution des trajectoires

Une petite API locale complète le prototype pour la démonstration. Elle charge le modèle du curriculum, exécute un épisode pour une forme connue et retourne les positions

successives des robots au format JSON. Une page Web peut alors dessiner les points de la cible et animer ces positions. L'interface n'exécute pas les règles de contrôle elle-même : le calcul est réalisé par le modèle PyTorch, puis seul le résultat est affiché dans le navigateur.

Ce choix évite de présenter comme résultat du GNN une animation construite avec des règles manuelles en JavaScript. Il ne s'agit pas d'un système de contrôle temps réel ni d'une communication par WebSocket. L'API est un outil de visualisation local, utile pour expliquer le modèle et rejouer un scénario déterminé par une graine.

3.8 Informations locales et information de mission

Le terme local peut recouvrir plusieurs réalités. Dans le contrôleur proposé, les informations provenant d'autres robots sont locales au sens strict : elles sont limitées aux voisins reliés dans le graphe de communication. Pour chaque arête, le message dépend des états des deux robots, de leur décalage relatif et de leur distance. Aucun message n'est envoyé directement par un robot situé hors de portée.

La consigne de forme est d'une autre nature. Elle est définie avant l'exécution de la mission et est partagée par l'essaim. Dans le curriculum réduit, un identifiant de forme est converti en représentation interne par le réseau. Dans la configuration complexe, le contrôleur reçoit en plus un vecteur local vers le point de contour le plus proche. Cet ajout facilite le suivi de formes détaillées, mais donne aussi une information géométrique plus riche. Les deux configurations ne doivent donc pas être confondues lors de l'analyse des résultats.

Cette séparation permet de répondre à une objection fréquente. Le système n'est pas centralisé parce qu'une forme est connue de tous : une carte de mission ou une balise peut être fournie avant le déploiement sans transmettre à chaque pas l'état complet de tous les robots. En revanche, le système ne serait plus strictement local si un serveur recalculait à chaque instant une affectation globale et la distribuait robot par robot. Cette seconde hypothèse n'est pas utilisée dans le simulateur.

3.9 Configurations d'entraînement

La première configuration entraîne vingt robots sur un cercle. Elle emploie une dimension cachée de 32, deux tours de passage de messages, une taille de lot de huit épisodes et 80 itérations. Un épisode comporte vingt pas de simulation. Ce réglage court a été choisi pour valider la cohérence du pipeline avant de lancer des campagnes plus coûteuses. Le modèle obtenu est conservé avec la courbe de perte et la configuration correspondante.

Le curriculum réduit reprend ensuite un modèle compatible et introduit progressivement trois géométries, trois chiffres, trois lettres et deux nombres. Chaque niveau conserve

les cibles précédentes afin de réduire le risque qu'un nouvel apprentissage efface complètement les comportements acquis. Cette stratégie n'élimine pas le problème de l'oubli catastrophique, mais elle évite d'entraîner chaque niveau uniquement sur la dernière forme ajoutée.

Une campagne complexe est également définie. Elle utilise cinquante robots, cinquante points de contour, une dimension cachée de 64, trois tours de messages et des épisodes de soixante pas. Elle est plus proche du niveau de détail requis pour les symboles, mais son coût sur processeur est plus élevé. Au moment de la rédaction, le niveau géométrique de cette campagne a été exécuté ; les niveaux suivants doivent encore être entraînés avant de pouvoir être analysés comme résultats complets.

3.10 Mesures utilisées pendant et après l'apprentissage

La distance de Chamfer est utilisée parce qu'elle compare deux ensembles de points sans imposer d'ordre ou d'affectation fixe. Le premier terme vérifie qu'aucun robot ne s'éloigne fortement de la cible ; le second vérifie qu'aucune partie de la cible ne reste trop loin de tous les robots. Elle n'est pas suffisante à elle seule. Deux configurations visuellement différentes peuvent avoir une distance voisine, notamment lorsqu'une zone fine est mal couverte.

Le projet complète donc cette distance par une pénalité de collision et par une mesure de régularité des distances au plus proche voisin. Le taux de collision décrit le nombre de paires qui deviennent trop proches au cours de l'épisode. La régularité renseigne sur la répartition de l'essaim : une valeur faible indique que les espacements sont plus homogènes, mais ne garantit pas que la forme soit correcte. Enfin, un taux de succès est calculé à partir d'un seuil explicite de distance. Ce seuil doit être affiché avec le résultat, car son choix influence directement l'interprétation du pourcentage de succès.

Pour les lettres et les nombres, une évaluation visuelle reste nécessaire dans cette première version. Le mémoire ne déclare pas qu'un caractère est lisible sur la seule base d'une faible distance de Chamfer. Une suite logique du travail serait de définir un protocole de lisibilité, par exemple avec des évaluateurs humains ou un classifieur séparé entraîné sur des rendus d'essaims.

3.11 Déroulement détaillé d'un épisode

Un épisode commence par le choix d'une cible et le tirage des positions initiales dans l'arène. Les vitesses sont initialisées à zéro. À chaque pas, le simulateur calcule toutes les distances inter-robots puis construit le graphe de communication en conservant uni-

quement les paires séparées par une distance inférieure au rayon fixé. Cette opération est répétée à chaque instant : un voisin au début de l'épisode peut ne plus être voisin après quelques déplacements.

Le contrôleur traite ensuite l'état de chaque robot. Il encode la vitesse, la consigne disponible et l'identifiant de la cible. Pour chaque arête du graphe, il construit un message à partir des états latents des deux extrémités et de leur position relative. Les messages reçus sont moyennés, l'état latent est mis à jour, puis le même mécanisme est répété selon le nombre de tours configuré. Un décodeur transforme enfin l'état final en une commande de vitesse à deux dimensions. La norme de cette commande est limitée avant la mise à jour de la position.

Pendant l'entraînement, la simulation est différentiable. Les opérations décrites ci-dessus sont réalisées avec des tenseurs PyTorch, ce qui permet de calculer la dérivée de la perte par rapport aux poids du GNN. Après la rétropropagation, l'optimiseur Adam modifie les paramètres. Pendant l'évaluation, les poids sont gelés et le même épisode fournit uniquement des mesures. Cette différence est importante : l'évaluation ne doit pas modifier le modèle après avoir vu les scénarios testés.

Le simulateur limite les positions aux frontières de l'arène. Ce choix empêche des valeurs non physiques de se propager dans la simulation, mais il peut créer un effet de bord : un robot qui atteint une limite ne se comporte pas comme dans un environnement ouvert. Les résultats sont donc interprétés dans une arène carrée finie, non comme une preuve de comportement dans tout plan euclidien.

3.12 Choix des hyperparamètres

Les hyperparamètres ne sont pas présentés comme des constantes universelles. Le rayon de communication définit le compromis entre localité et connectivité. S'il est trop faible, le graphe se fragmente et les robots isolés ne reçoivent plus de messages. S'il est trop élevé, tous les robots deviennent voisins et l'hypothèse de communication locale perd son intérêt. La valeur de 2,5 utilisée dans les premières expériences est un choix de simulation à documenter, pas une distance radio correspondant à une plateforme précise.

La profondeur du GNN règle le nombre de tours de messages disponibles au sein d'un pas de simulation. Deux tours sont employés dans le curriculum réduit. Cette valeur permet à un robot d'intégrer indirectement des informations provenant de voisins de voisins, tout en gardant un modèle léger. La campagne complexe utilise trois tours, car les contours demandent une coordination plus fine. Une profondeur supérieure pourrait étendre l'information disponible, mais son effet doit être mesuré : elle augmente le nombre de calculs et peut réduire la stabilité de l'apprentissage.

La dimension cachée contrôle la capacité du modèle à représenter les états locaux et les messages. La première campagne utilise 32 composantes latentes, ce qui limite le

nombre de paramètres et facilite les essais rapides. La campagne complexe passe à 64 composantes. Cette augmentation ne constitue pas une preuve qu'un modèle plus grand est meilleur ; elle répond au besoin de représenter des formes avec davantage de points et des épisodes plus longs. Une étude complète devrait comparer plusieurs dimensions à budget d'entraînement identique.

La vitesse maximale et le pas de temps sont liés. Une vitesse élevée ou un pas trop grand permet aux robots de rejoindre rapidement la cible, mais peut provoquer des dépassements et des collisions. À l'inverse, un pas trop petit rend l'épisode long et peut empêcher la forme de se construire dans la durée choisie. Les pénalités de mouvement et de collision servent à régulariser ce compromis. Les valeurs retenues sont conservées dans les fichiers de configuration afin que chaque résultat puisse être interprété avec son contexte numérique.

3.13 Gestion des formes et du curriculum

Le jeu de données de formes est séparé du modèle. Il contient des images et les nuages de points associés. Cette séparation a deux avantages. D'une part, la génération ou la correction d'une cible n'oblige pas à modifier l'architecture du GNN. D'autre part, elle permet de définir explicitement quels fichiers appartiennent à l'entraînement, à la validation ou au test. Cette discipline devient indispensable dès que l'on veut déclarer qu'un modèle généralise à des formes non vues.

Le curriculum utilisé dans le projet commence par les géométries simples. Le modèle voit ensuite les chiffres, puis quelques lettres et nombres. Le principe n'est pas que les premières formes soient intrinsèquement plus importantes ; elles fournissent un problème initial plus stable. Un cercle, un carré et un triangle permettent notamment de vérifier que la perte de forme et la pénalité de collision entraînent les mouvements dans une direction utile avant d'ajouter des contours plus irréguliers.

Chaque nouveau niveau conserve les cibles des niveaux précédents. Sans cette précaution, le modèle pourrait optimiser fortement les nouvelles formes et perdre la capacité acquise sur le cercle ou le triangle. Cette conservation réduit le risque d'oubli, mais elle ne le supprime pas. Elle augmente également la diversité des épisodes et peut ralentir l'amélioration sur une cible particulière. Ces compromis sont normaux dans un apprentissage progressif et doivent être visibles dans les courbes et les métriques par forme.

Chapitre 4

Expériences et résultats

4.1 Protocole pilote

Les expériences sont organisées en deux phases. La première contrôle que le GNN peut être entraîné sur un cercle et qu'il produit visuellement une distribution autour de la cible. La seconde est un protocole pilote d'évaluation, conçu pour vérifier la chaîne de mesure avant une campagne plus large. Ce pilote fixe dix graines par condition, soit 240 épisodes. Il évalue une cible circulaire avec 20, 40 et 100 robots, ainsi que des retraits de 5, 10 et 20% des robots. Chaque épisode dure 20 pas.

Les métriques rapportées sont la distance de Chamfer, le nombre ou le taux de collisions, la régularité des distances entre voisins et un taux de succès associé à un seuil de distance de Chamfer de 0,5. La distance de Chamfer mesure l'adéquation spatiale à la forme, mais elle ne garantit ni une couverture uniforme ni la lisibilité d'un symbole. Les métriques sont donc complémentaires et doivent être interprétées avec les visualisations finales.

Le protocole distingue deux types de comparaison. La comparaison entre le GNN et la règle de regroupement vérifie que le contrôleur entraîné atteint mieux la cible circulaire dans le cadre retenu. Les variations de taille et de pannes évaluent ensuite le même GNN dans des conditions différentes de celles de l'entraînement. Le protocole ne permet pas encore de séparer l'effet de tous les choix de conception : il manque en particulier une baseline apprise sans communication et une règle manuelle recevant la même consigne de forme.

Avant toute agrégation, chaque épisode est reproductible à partir d'une graine. Les positions initiales sont tirées aléatoirement dans l'arène, puis la même dynamique est appliquée pendant vingt pas. Pour un scénario de panne, le retrait se produit à mi-épisode. Les fichiers bruts conservent une ligne par épisode ; le tableau de synthèse rapporte ensuite la moyenne et l'écart-type de la distance de forme. Cette organisation permet de retrouver les cas individuels qui expliquent une moyenne inhabituelle.

4.2 Résultats préliminaires

Pour l'entraînement initial sur un cercle, la perte observée avec la graine 41 passe de 9,84 à 0,28 en 80 itérations. Cette diminution indique que l'optimisation est fonctionnelle dans le simulateur ; elle ne constitue pas une mesure de généralisation, car elle concerne une configuration et une graine particulières.

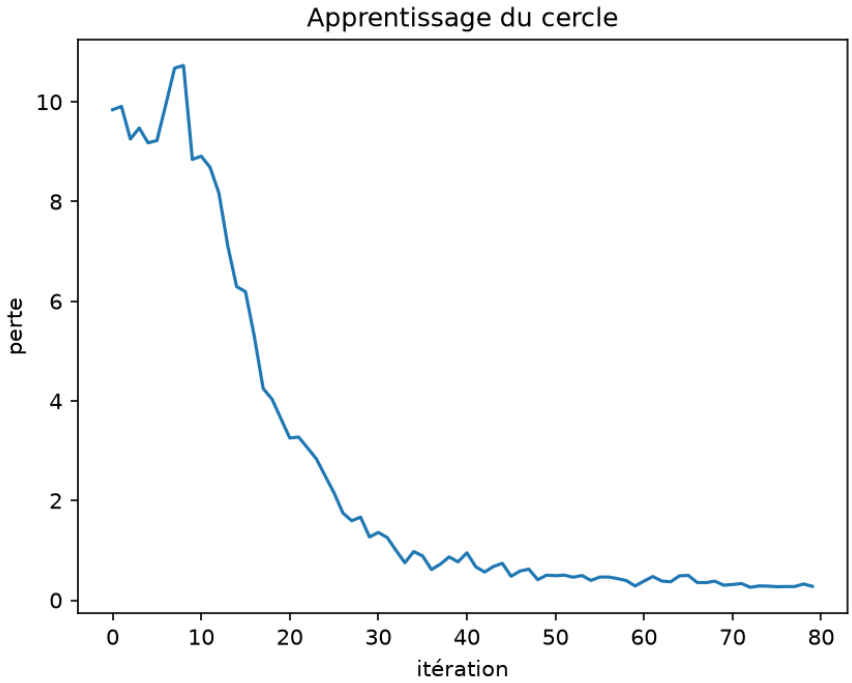


FIGURE 4.1 – Évolution de la perte pendant l'entraînement initial du cercle, pour la graine 41. Cette courbe sert de diagnostic d'optimisation et non d'intervalle de confiance.

Le tableau 4.1 présente les résultats du pilote pour 20 robots. Sans panne, le GNN atteint une distance de Chamfer moyenne de 0,607, contre 3,956 pour la règle de regroupement. Après retrait de 20% des robots, les valeurs sont respectivement 0,644 et 4,162. Dans les mêmes conditions, le taux de collisions mesuré est de 0,031 pour le GNN sans panne et de 0,018 après retrait de 20% des robots ; la règle de regroupement présente des valeurs de 0,187 et 0,129.

TABLE 4.1 – Résultats du protocole pilote à 20 robots (10 graines par condition).

Contrôleur	Pannes	Chamfer moyen	Taux de collisions	Succès
GNN	0%	0,607	0,031	0,20
GNN	20%	0,644	0,018	0,00
Regroupement	0%	3,956	0,187	0,00
Regroupement	20%	4,162	0,129	0,00

Cette comparaison doit être interprétée avec prudence. La règle de regroupement ne reçoit pas de consigne explicite de cercle : elle répond donc à un problème d'information

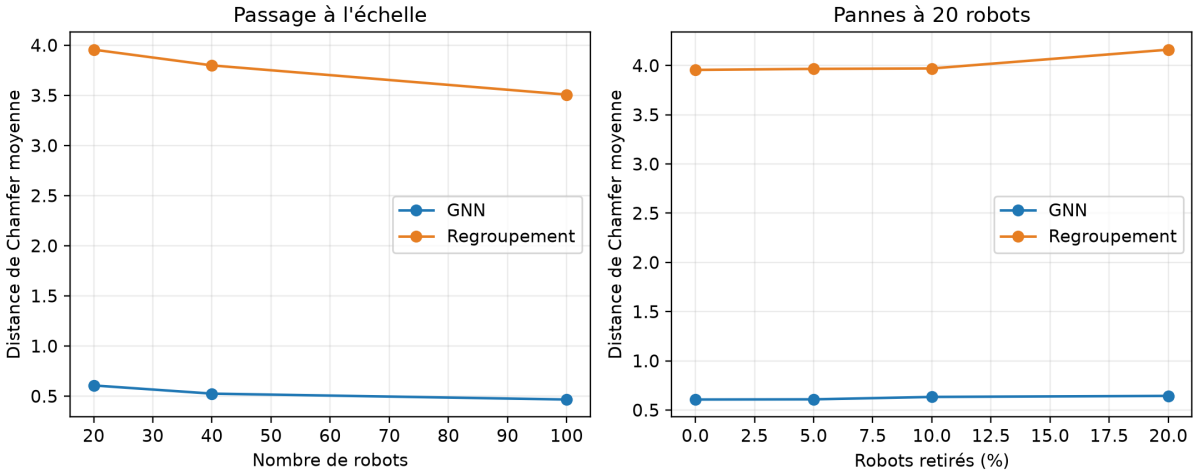


FIGURE 4.2 – Synthèse du protocole pilote calculée à partir de 240 épisodes. À gauche, l'effet de la taille de l'essaim sans panne ; à droite, l'effet des pannes avec 20 robots. Une valeur plus faible de distance de Chamfer indique une meilleure adéquation à la cible circulaire.

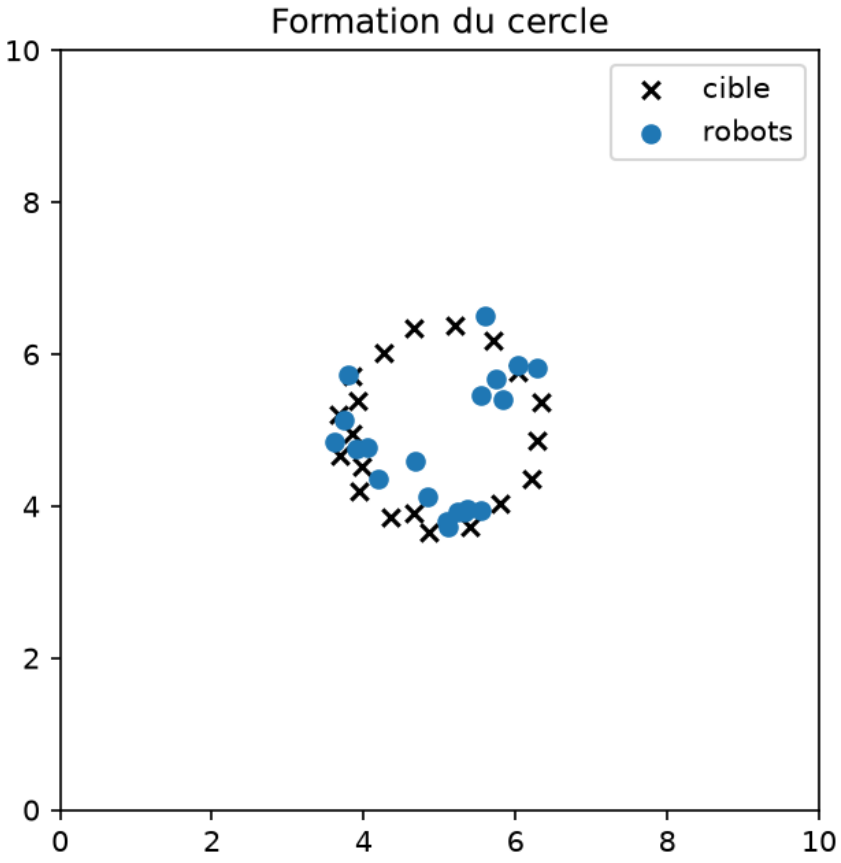


FIGURE 4.3 – Configuration finale obtenue pour l'entraînement du cercle. Les robots représentent l'approximation discrète de la cible par l'essaim.

différent. Le tableau montre que le contrôleur entraîné est plus proche de la forme circulaire dans ce protocole, mais ne démontre pas qu'il est supérieur à toute règle décentralisée disposant de la même consigne.

4.3 Effet du nombre de robots

Le même modèle est exécuté avec 20, 40 et 100 robots, alors que son entraînement de référence a été réalisé avec 20 robots. Cette expérience vérifie d'abord une propriété pratique : la politique peut être évaluée avec une taille différente sans modifier le nombre de paramètres. Elle vérifie ensuite une propriété empirique : la qualité obtenue reste-t-elle acceptable lorsque le graphe de communication devient plus dense et que la cible doit être couverte par davantage de robots ?

Dans le pilote sans panne, la distance moyenne du GNN est de 0,607 avec 20 robots, de 0,525 avec 40 robots et de 0,467 avec 100 robots. La règle de regroupement reste nettement plus éloignée de la cible, avec des distances respectives de 3,956, 3,799 et 3,507. La diminution observée pour le GNN peut venir d'une meilleure couverture de l'échantillonnage circulaire lorsque davantage de points sont disponibles. Elle ne signifie pas nécessairement que la politique est meilleure dans tous les régimes de densité : les cibles sont redimensionnées pour conserver le même nombre de points que les robots, et le protocole reste limité à dix épisodes par condition.

Le tableau 4.2 récapitule ces résultats. Il doit être lu comme un contrôle de bon fonctionnement à taille variable. Une validation plus ambitieuse demanderait de répéter l'entraînement avec plusieurs graines, d'étudier plusieurs rayons de communication et de vérifier la conservation de la connectivité pendant toute la trajectoire.

TABLE 4.2 – Passage à l'échelle sans panne dans le protocole pilote, avec dix épisodes par condition.

Contrôleur	Robots	Chamfer moyen	Taux de succès
GNN	20	0,607	0,20
GNN	40	0,525	0,60
GNN	100	0,467	1,00
Regroupement	20	3,956	0,00
Regroupement	40	3,799	0,00
Regroupement	100	3,507	0,00

La figure 4.4 illustre l'évolution de la distance avec le taux de pannes. Le faible écart observé entre 0% et 20% de pannes pour le GNN est une indication encourageante dans ce cadre précis, mais la taille de l'échantillon ne permet pas d'en déduire un intervalle de confiance robuste. Une campagne finale devra employer au moins 50 répétitions par condition et rapporter la variabilité entre graines.

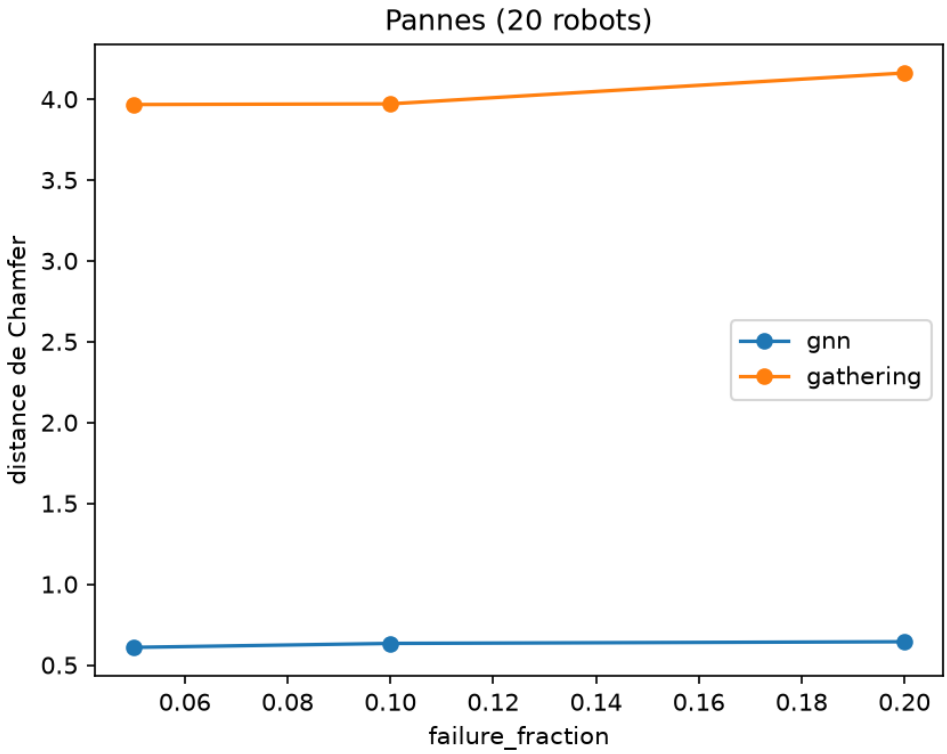


FIGURE 4.4 – Distance moyenne de Chamfer en fonction du taux de pannes pour 20 robots dans le protocole pilote.

4.4 Lecture des scénarios de panne

Dans le simulateur, une panne est introduite à mi-parcours : une fraction des robots actifs est retirée aléatoirement. Les robots retirés ne contribuent plus aux messages et ne sont plus pris en compte dans la mesure finale de la forme. Cette définition correspond à une disparition propre d'agents du réseau. Elle permet d'étudier comment le reste de l'essaim se réorganise, mais elle est moins sévère qu'une panne matérielle réelle, où un robot peut rester immobile, produire des mesures incohérentes ou gêner physiquement ses voisins.

Le résultat doit donc être formulé avec précision. Le GNN conserve, dans le pilote, une distance de Chamfer plus faible que la baseline après le retrait de 5, 10 ou 20% des robots. Cela montre que la politique n'est pas limitée à une composition rigide de vingt agents. Il ne s'agit pas encore d'une démonstration d'auto-réparation générale. Le protocole n'observe ni le temps nécessaire au retour vers la forme, ni la connectivité du graphe, ni le comportement d'un robot défaillant qui resterait dans l'arène.

4.5 Curriculum de formes

Le curriculum réduit entraîne successivement les niveaux géométrie, chiffres, symboles et nombres. Les pertes finales enregistrées sont respectivement 0,318, 0,248, 0,205 et 0,156.

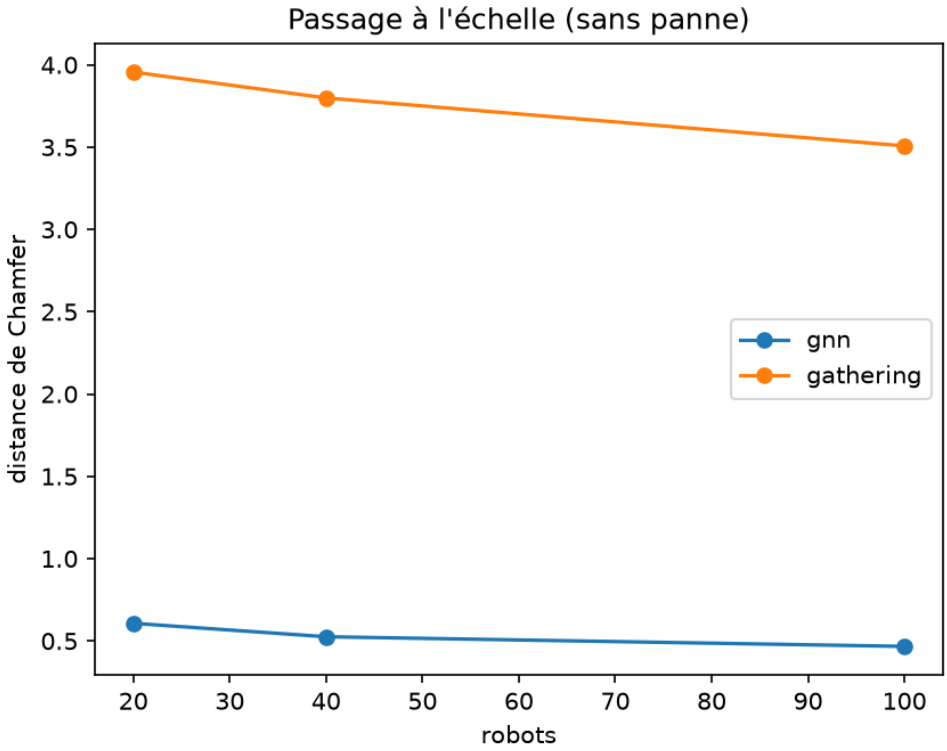


FIGURE 4.5 – Distance moyenne de Chamfer lors du passage à l'échelle dans le protocole pilote. Cette figure doit être complétée par les valeurs numériques et leur dispersion dans la campagne finale.

Cette baisse ne doit pas être comparée directement entre niveaux, car les ensembles de cibles et l'état initial du modèle évoluent. Elle confirme surtout que la chaîne d'entraînement peut poursuivre l'optimisation lorsque de nouvelles formes sont ajoutées.

Les visualisations montrent une couverture approximative des cibles symboliques. En l'état, les lettres et les nombres ne sont pas suffisamment lisibles pour soutenir une affirmation de reconnaissance ou de formation précise. L'amélioration de cette lisibilité exigera un protocole visuel et quantitatif dédié, davantage d'itérations, ainsi qu'une étude de l'effet du nombre de robots et de la densité de points de la forme.

4.6 Exemples, résultats négatifs et limites de lecture

Les exemples visuels ont une place importante dans ce projet parce que les métriques globales ne montrent pas tous les défauts d'une formation. Une configuration peut avoir une distance moyenne faible alors qu'un angle de lettre est mal dessiné ou que plusieurs robots forment un petit amas. C'est pourquoi les planches de visualisation présentent à la fois les points de la cible et la position finale des robots. Elles permettent de repérer rapidement les zones non couvertes, les espacements irréguliers et les formes qui restent ambiguës.

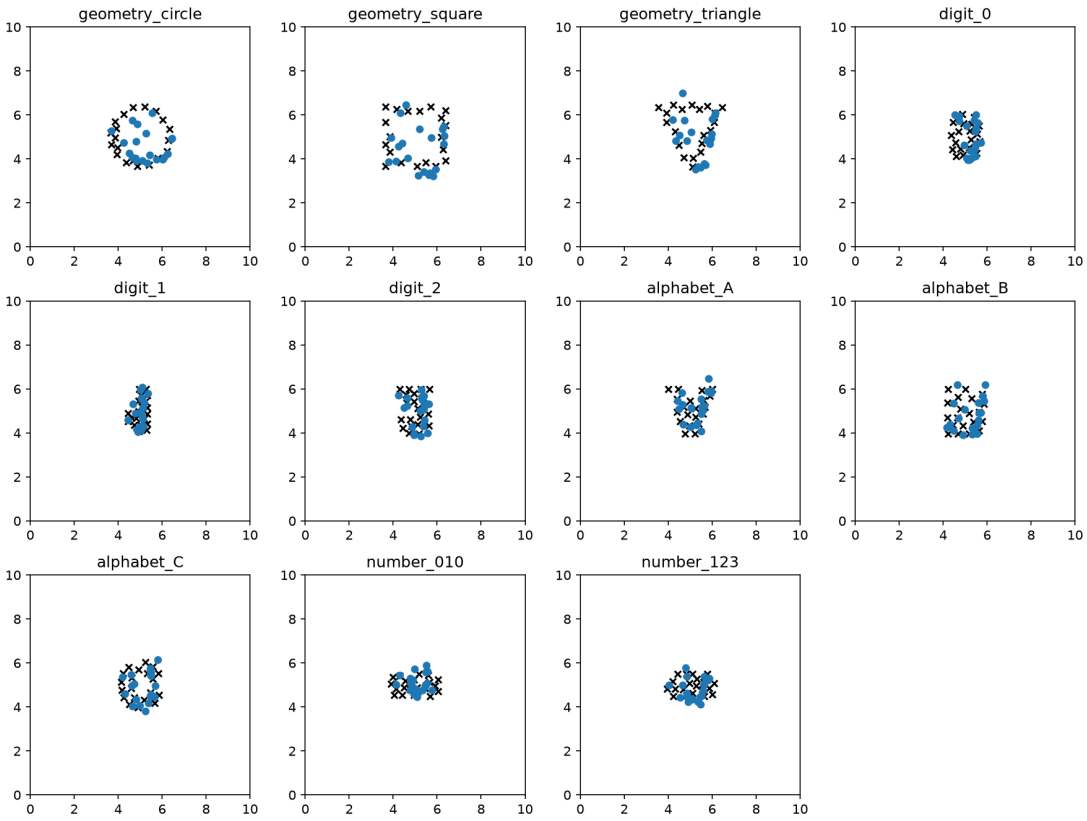


FIGURE 4.6 – Planche de visualisation issue de la dernière étape du curriculum réduit. Elle illustre des formations approximatives et ne constitue pas encore une validation de lisibilité des symboles.

Ces images ne doivent pas être sélectionnées uniquement parmi les meilleurs épisodes. Pour chaque campagne finale, il faudra conserver un ensemble fixe de graines, afficher au moins un exemple favorable et un exemple défavorable, puis relier ces cas aux statistiques. Cette procédure est particulièrement importante pour les symboles : une valeur moyenne ne peut pas établir à elle seule qu'une lettre est lisible par une personne.

Le curriculum réduit donne donc un résultat de faisabilité. Il montre que l'entraînement peut être poursuivi lorsqu'on ajoute de nouvelles cibles. Il ne suffit pas à dire que le modèle a appris un alphabet ou qu'il généralise à toutes les combinaisons de chiffres. Les lettres A, B et C ainsi que les nombres 010 et 123 sont les seules cibles symboliques de cette campagne. Les autres formes générées par le projet font partie du jeu de données, mais ne sont pas automatiquement des formes connues du modèle final.

4.7 Discussion des résultats

Le premier résultat est la validation de la chaîne expérimentale : le simulateur, le contrôleur à messages, la fonction de perte et l'optimisation permettent d'obtenir une formation circulaire plus proche de la cible que l'état initial. Le partage des paramètres permet d'appliquer la même politique à chaque robot, tandis que l'agrégation locale accepte un nombre variable de voisins. Le protocole pilote indique aussi que le GNN est plus proche du cercle que la règle de regroupement utilisée comme contrôle.

Cette observation doit rester limitée à son cadre. La règle de regroupement ne reçoit pas la même consigne de forme que le GNN. Elle répond donc à un problème différent et ne peut pas servir de preuve qu'un GNN est meilleur que toute règle décentralisée. Une comparaison plus juste devra inclure une règle classique ayant accès à la même cible, ainsi qu'une version apprise sans échange de messages.

Les résultats sur les pannes sont également préliminaires. La distance moyenne de Chamfer augmente peu entre les scénarios sans panne et les scénarios avec retrait de robots, mais le taux de réussite dépend du seuil choisi et tombe à zéro dans certaines conditions. Il serait donc imprudent de conclure à une robustesse générale. Pour répondre correctement à cette question, il faut multiplier les graines, calculer des intervalles de confiance et étudier les cas où l'essaim se fragmente ou couvre mal une partie de la cible.

Les limites de simulation doivent aussi être rappelées. Les robots possèdent un repère commun, les positions sont exactes et les messages ne subissent ni délai ni perte. La collision est pénalisée par une fonction continue plutôt que par un modèle de contact. Enfin, les lettres et les nombres issus du curriculum réduit restent des formations approximatives : leur présence dans une planche ne démontre ni une reconnaissance fiable ni une généralisation à des symboles non vus.

La suite expérimentale doit donc consolider le protocole plutôt que l'élargir trop vite. Les priorités sont cinquante répétitions ou plus par condition, une baseline sans commu-

nication, des contrôleurs classiques recevant une consigne équivalente, du bruit de mesure et une réduction progressive de la portée de communication. Ces expériences permettront de distinguer ce qui relève réellement de l'échange de messages appris de ce qui provient de la consigne géométrique fournie au système.

4.8 Lien entre agrégation et résultats observés

L'agrégation moyenne rend le contrôleur compatible avec des voisinages de taille variable. Cette propriété est particulièrement pertinente dans les expériences de passage à l'échelle : lorsque le nombre de robots augmente, les robots situés au centre de l'arène peuvent recevoir plus de messages que ceux situés près du bord. Une somme brute de messages ferait augmenter la norme de l'entrée agrégée avec cette densité. La moyenne évite ce changement d'échelle immédiat, mais elle ne garantit pas à elle seule la qualité de formation.

Les résultats à 40 et 100 robots montrent que l'architecture ne s'effondre pas lorsque le nombre de nœuds change. Cette observation est cohérente avec le partage des paramètres et la moyenne des messages. Elle doit néanmoins être interprétée avec prudence : le succès accru à 100 robots peut aussi s'expliquer par la densité plus importante de points disponibles sur un même contour. Une ablation comparant moyenne, somme normalisée et absence de messages serait nécessaire pour attribuer précisément l'amélioration à la stratégie d'agrégation.

Le rapport avec le flocking est également limité mais instructif. La règle manuelle de regroupement utilise une interaction locale simple et produit une cohésion sans représenter explicitement le cercle. Le GNN reçoit une consigne supplémentaire et apprend ses réponses à partir d'exemples simulés. La comparaison actuelle montre donc la différence entre un comportement de rassemblement et un comportement conditionné par une cible, pas une opposition définitive entre méthodes manuelles et méthodes apprises.

4.9 Analyse des métriques

La distance de Chamfer est la métrique principale car elle répond directement à la question de la proximité entre la formation et la cible. Une valeur faible signifie que, en moyenne, les robots sont proches d'un point du contour et que les points du contour sont proches d'un robot. Elle ne décrit cependant pas la dynamique. Deux épisodes peuvent produire la même distance finale alors que l'un atteint la cible rapidement et que l'autre oscille jusqu'au dernier pas. Le temps de convergence est donc une mesure complémentaire prévue pour les expériences finales.

Le taux de collision doit également être interprété avec soin. Dans cette étude, il est calculé à partir d'un seuil de sécurité et des positions simulées. Il ne mesure pas une

collision mécanique avec contact, car le simulateur ne comporte pas de modèle de masse, de friction ou de rebond. Son rôle est de signaler les trajectoires qui placent trop souvent deux robots à une distance incompatible avec une marge de sécurité définie. Une baisse du taux de collision est favorable, mais elle ne compense pas une formation qui manquerait largement la cible.

La régularité des distances au plus proche voisin complète les deux mesures précédentes. Une formation peut être proche de la cible tout en laissant une partie du contour très dense et une autre presque vide. Cette métrique détecte une partie de ce phénomène, sans fournir une mesure parfaite de couverture. Dans la campagne finale, il serait utile de présenter les métriques ensemble dans un tableau par condition, plutôt que de sélectionner celle qui donne le résultat le plus favorable.

4.10 Menaces à la validité

La première menace concerne le nombre limité de répétitions. Dix épisodes par condition permettent de vérifier que les scripts fonctionnent et de détecter un écart important entre deux méthodes. Ils ne suffisent pas pour estimer précisément une moyenne, comparer des distributions ou établir un intervalle de confiance robuste. La cible de cinquante répétitions par condition est retenue pour la suite parce qu'elle rendra les agrégats plus informatifs, même si elle ne remplace pas une analyse statistique complète.

La deuxième menace est l'écart entre le simulateur et un système réel. Les robots du projet connaissent un repère commun, échangent des messages exacts et sont modélisés par une vitesse directement appliquée. Dans un robot physique, les estimations de position sont bruitées, les roues ont une inertie et les communications peuvent être retardées. Ces hypothèses rendent l'étude actuelle plus contrôlable, mais limitent la portée des conclusions.

La troisième menace concerne les formes elles-mêmes. Les nombres et lettres du curriculum réduit sont peu nombreux, et ils sont utilisés pendant l'entraînement. La capacité à former A, B ou C ne prouve pas une généralisation à toutes les lettres. Une expérience de généralisation devra réserver des formes complètes avant l'entraînement, puis mesurer la performance sur ces cibles non vues sans modifier les poids du modèle.

4.11 Plan d'expériences pour la campagne finale

La campagne finale doit être organisée autour de questions simples et mesurables. La première question concerne la qualité de formation : pour chaque catégorie de formes, quelle est la distance à la cible, quelle est la régularité de la répartition et quelle proportion d'épisodes franchit le seuil de succès ? La deuxième question concerne l'apport

de la communication : que se passe-t-il lorsqu'on retire les messages de voisins en conservant la même consigne de forme ? La troisième question concerne la robustesse : comment évoluent les mesures lorsque des robots disparaissent, lorsque le rayon de communication diminue ou lorsque les positions sont bruitées ?

Chaque condition doit être fixée avant de consulter les résultats. Par exemple, une expérience de panne peut combiner trois tailles d'essaim, quatre fractions de retrait et cinquante graines. Les mêmes positions initiales peuvent être réutilisées pour comparer deux contrôleurs, afin que l'écart observé ne soit pas causé par une série de départs plus faciles pour l'une des méthodes. Les données brutes doivent ensuite être conservées pour permettre le recalcul des moyennes, écarts-types et graphiques.

Une présentation complète devra associer un tableau numérique à une figure. Le tableau donne les valeurs précises et leur dispersion. La figure montre la tendance quand un facteur varie, par exemple le pourcentage de pannes. Les images finales et les animations jouent un troisième rôle : elles permettent de comprendre la nature spatiale d'un échec. Cette complémentarité évite de réduire un comportement collectif à un seul nombre.

4.12 Interprétation prudente des résultats pilotes

Le pilote est utile parce qu'il valide l'enchaînement complet des scripts. Il produit des épisodes, agrège les résultats et génère des graphiques sans intervention manuelle. Il met aussi en évidence un écart important entre le GNN conditionné par la cible et la règle de regroupement. Cet écart justifie de poursuivre les expériences, mais il ne suffit pas pour une conclusion forte sur la supériorité d'une méthode.

La première raison est l'asymétrie d'information déjà discutée. La seconde est le nombre de répétitions. La troisième est que les résultats concernent principalement une cible circulaire. Une méthode peut être très performante pour un contour régulier et moins convaincante pour une lettre présentant des angles ou des traits éloignés. L'objectif de la campagne finale est donc de remplacer une affirmation générale par un ensemble de résultats précis, chacun associé à une forme, une taille d'essaim et une perturbation donnée.

Cette prudence améliore la valeur du mémoire. Un résultat négatif, comme une lettre difficile à couvrir ou un effondrement lorsque la portée devient trop faible, n'est pas un échec de rédaction. Il renseigne sur les limites de la politique et aide à déterminer quelles observations ou quels mécanismes doivent être ajoutés. Dans un travail sur les systèmes distribués, expliquer pourquoi une méthode cesse de fonctionner est souvent aussi utile que montrer ses meilleurs exemples.

Chapitre 5

Conclusion et perspectives

Ce travail a établi une chaîne expérimentale reproductible pour étudier la formation de formes par un essaim contrôlé par GNN. Le système associe un simulateur 2D, des formes représentées par nuages de points, une politique locale à paramètres partagés et un apprentissage différentiable. Le cercle constitue une première validation visuelle et quantitative du fonctionnement de cette chaîne. Les modules de génération de données, d'entraînement, d'évaluation et de visualisation sont séparés afin que les expériences puissent être rejouées à partir de leurs configurations.

Le résultat principal n'est pas encore une démonstration de déploiement sur robots physiques. Il est plus précis : dans le simulateur retenu, une politique GNN partagée peut recevoir des informations locales, échanger des messages avec ses voisins et produire des mouvements qui rapprochent l'essaim d'une cible. Le modèle se laisse également exécuter avec un nombre de robots différent de celui utilisé pendant l'entraînement. Cette propriété découle du partage des paramètres et de l'agrégation des messages, mais sa qualité doit rester vérifiée expérimentalement pour chaque scénario.

Le protocole pilote suggère que le contrôleur appris atteint une meilleure adéquation à une cible circulaire qu'une règle locale de regroupement, y compris dans les scénarios de retrait étudiés. Cette conclusion est volontairement restreinte : le pilote ne comporte que dix graines par condition et la baseline ne dispose pas de la même consigne. Les formes symboliques sont, elles aussi, encore au stade de couverture approximative. Elles montrent que le pipeline traite plusieurs types de contours ; elles ne permettent pas encore d'affirmer une formation lisible et généralisable de lettres ou de nombres.

Les premières perspectives sont expérimentales. Il faut augmenter le nombre de répétitions, fixer un jeu de formes non vues, mesurer des intervalles de confiance et conserver les épisodes favorables comme les épisodes défavorables. Il faut aussi ajouter des baselines équitables : une politique sans communication, une règle manuelle recevant une consigne comparable et plusieurs profondeurs de passage de messages. Ces comparaisons permettront de mesurer l'apport spécifique de l'agrégation locale plutôt que l'effet général de l'information de cible.

Les perspectives techniques concernent le réalisme du simulateur. Des bruits de position et de communication, des délais, des obstacles et une dynamique de robot non holonome rapprocheraient le cadre de situations réelles. À plus long terme, la consigne géométrique pourrait être fournie par une perception locale, une balise ou une carte distribuée. Le passage sur une plateforme physique demanderait alors une estimation de pose, une stratégie de sécurité et une validation progressive, d'abord sur quelques robots puis sur un essaim plus grand.

Le projet répond donc partiellement à la question initiale. Il montre la faisabilité d'un contrôleur GNN local pour une formation de formes en simulation et fournit les outils nécessaires à une évaluation plus rigoureuse. La suite du travail devra déterminer, avec des expériences mieux contrôlées, dans quelles conditions cette approche apporte un avantage réel par rapport aux méthodes de contrôle distribué classiques.

Bibliographie

[1] Peter W. Battaglia et al. “Relational Inductive Biases, Deep Learning, and Graph Networks”. In : *arXiv preprint arXiv :1806.01261* (2018).

[2] Manuele Brambilla et al. “Swarm robotics : a review from the swarm engineering perspective”. In : *Swarm Intelligence* 7 (2013), p. 1-41. doi : 10.1007/s11721-012-0075-2.

[3] Fernando Gama, Ekaterina Tolstaya et Alejandro Ribeiro. “Graph Neural Networks for Decentralized Controllers”. In : *arXiv preprint arXiv :2003.10280* (2020).

[4] Fernando Gama et al. “Synthesizing Decentralized Controllers with Graph Neural Networks and Imitation Learning”. In : *IEEE Transactions on Signal Processing* 70 (2022), p. 202-215. doi : 10.1109/TSP.2021.3139935.

[5] Justin Gilmer et al. “Neural Message Passing for Quantum Chemistry”. In : *ICML*. 2017, p. 1263-1272.

[6] William L. Hamilton, Rex Ying et Jure Leskovec. “Inductive Representation Learning on Large Graphs”. In : *Advances in Neural Information Processing Systems*. T. 30. 2017. url : https://arxiv.org/abs/1706.02216.

[7] M. Ani Hsieh, Vijay Kumar et Luiz Chaimowicz. “Decentralized controllers for shape generation with robotic swarms”. In : *Robotica* 26.5 (2008), p. 691-701. doi : 10.1017/S0263574708004323.

[8] Chao Jiang, Xinchi Huang et Yi Guo. “End-to-end decentralized formation control using a graph neural network-based learning method”. In : *Frontiers in Robotics and AI* 10 (2023). doi : 10.3389/frobt.2023.1285412.

[9] Guannan Li et al. “Decentralized progressive shape formation with robot swarms”. In : *Autonomous Robots* 43.6 (2019), p. 1505-1521. doi : 10.1007/s10514-018-9807-5.

[10] Reza Olfati-Saber. “Flocking for Multi-Agent Dynamic Systems : Algorithms and Theory”. In : *IEEE Transactions on Automatic Control* 51.3 (2006), p. 401-420. doi : 10.1109/TAC.2005.864190.

[11] Craig W. Reynolds. “Flocks, Herds, and Schools : A Distributed Behavioral Model”. In : *SIGGRAPH*. 1987, p. 25-34. doi : 10.1145/37401.37406.

[12] Ekaterina Tolstaya et al. “Learning Decentralized Controllers for Robot Swarms with Graph Neural Networks”. In : *CoRL*. 2020, p. 671-682.